

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
import pandas as pd
import numpy as np
df = pd.read_csv('/content/drive/MyDrive/sentiment_project/1
                  encoding='latin-1',
                  header=None,
                  names=['target', 'id', 'date', 'flag', 'use

df = df.sample(n=100000, random_state=42)
df = df[['text', 'target']]
df['target'] = df['target'].replace(4, 1)

print(df.shape)
print(df['target'].value_counts())
```

```
(100000, 2)
target
1      50057
0      49943
Name: count, dtype: int64
```

```
import re
import nltk
nltk.download('stopwords', quiet=True)
from nltk.corpus import stopwords

# Load default stopwords
default_stopwords = set(stopwords.words('english'))

# Words to KEEP (remove from stopwords list)
negation_words = {
    "no", "not", "nor", "neither",
    "never", "nobody", "nothing", "nowhere",
    "don't", "didn't", "doesn't", "wouldn't",
    "couldn't", "shouldn't", "wasn't", "weren't",
    "isn't", "aren't", "won't", "can't", "cannot"
}

# Final stopwords = default minus negation words
stop_words = default_stopwords - negation_words
```

```
print(f"Default stopwords count: {len(default_stopwords)}")
print(f"Negation words kept: {len(negation_words)}")
print(f"Final stopwords count: {len(stop_words)}")
```

Default stopwords count: 198
 Negation words kept: 21
 Final stopwords count: 184

```
def clean_tweet(text):
    text = text.lower()
    text = re.sub(r'http\S+|www\S+', '', text)
    text = re.sub(r'@\w+', '', text)
    text = re.sub(r'#\w+', '', text)
    text = re.sub(r'^a-z\s]', '', text)
    words = text.split()
    words = [w for w in words if w not in stop_words]
    return ' '.join(words)
```

```
df['clean_text'] = df['text'].apply(clean_tweet)
print("Cleaning done!")
```

Cleaning done!

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer_improved = vectorizer = TfidfVectorizer(max_featu
X = vectorizer_improved.fit_transform(df['clean_text'])
y = df['target']

print("X shape:", X.shape)
print("y shape:", y.shape)
```

X shape: (100000, 8000)
 y shape: (100000,)

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_s
                                                    random

print("Training size:", X_train.shape)
print("Testing size:", X_test.shape)
```

Training size: (80000, 8000)
 Testing size: (20000, 8000)

```

import tensorflow as tf
from tensorflow import keras
from keras import layers

# Build improved model
ann_improved = keras.Sequential([
    layers.Dense(64, activation='relu', input_shape=(8000,)),
    layers.Dropout(0.5),
    layers.Dense(32, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(1, activation='sigmoid')
])

# Compile
ann_improved.compile(optimizer='adam',
                    loss='binary_crossentropy',
                    metrics=['accuracy'])

ann_improved.summary()

```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_3 (Dense)	(None, 64)	512,064
dropout_2 (Dropout)	(None, 64)	0
dense_4 (Dense)	(None, 32)	2,080
dropout_3 (Dropout)	(None, 32)	0
dense_5 (Dense)	(None, 1)	33

Total params: 514,177 (1.96 MB)
 Trainable params: 514,177 (1.96 MB)
 Non-trainable params: 0 (0.00 MB)

```

from keras.callbacks import EarlyStopping

# Early stopping
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=3,
    restore_best_weights=True
)

# Convert to dense
X_train_dense = X_train.toarray()
X_test_dense = X_test.toarray()

```

```
# Train
history = ann_improved.fit(
    X_train_dense, y_train,
    epochs=10,
    batch_size=64,
    validation_split=0.1,
    callbacks=[early_stop],
    verbose=1
)
```

```
Epoch 1/10
1125/1125 ————— 16s 13ms/step - accuracy: 0.6816 - loss: 0.5916 - val_accuracy: 0.7731 - val_loss: 0.4800
Epoch 2/10
1125/1125 ————— 13s 12ms/step - accuracy: 0.7965 - loss: 0.4498 - val_accuracy: 0.7742 - val_loss: 0.4794
Epoch 3/10
1125/1125 ————— 13s 12ms/step - accuracy: 0.8200 - loss: 0.4104 - val_accuracy: 0.7739 - val_loss: 0.4901
Epoch 4/10
1125/1125 ————— 13s 12ms/step - accuracy: 0.8408 - loss: 0.3741 - val_accuracy: 0.7695 - val_loss: 0.5045
Epoch 5/10
1125/1125 ————— 13s 12ms/step - accuracy: 0.8573 - loss: 0.3453 - val_accuracy: 0.7701 - val_loss: 0.5229
```

```
# Evaluate on test set
test_loss, test_accuracy = ann_improved.evaluate(X_test_dense)
print(f"Test Accuracy: {round(test_accuracy * 100, 2)}%")
print(f"Test Loss: {round(test_loss, 4)}")
```

```
Test Accuracy: 76.81%
Test Loss: 0.4831
```

```
from sklearn.metrics import classification_report
```

```
# Classification report
y_pred_prob = ann_improved.predict(X_test_dense, verbose=0)
y_pred = (y_pred_prob > 0.5).astype(int)

print("Improved ANN Classification Report:")
print(classification_report(y_test, y_pred))
```

```
Improved ANN Classification Report:
              precision    recall  f1-score   support

     0       0.77         0.76         0.77         9995
     1       0.76         0.78         0.77        10005

 accuracy                   0.77         20000
 macro avg       0.77         0.77         0.77         20000
 weighted avg    0.77         0.77         0.77         20000
```

```
import matplotlib.pyplot as plt
```

```
# Plot accuracy and loss curves
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
```

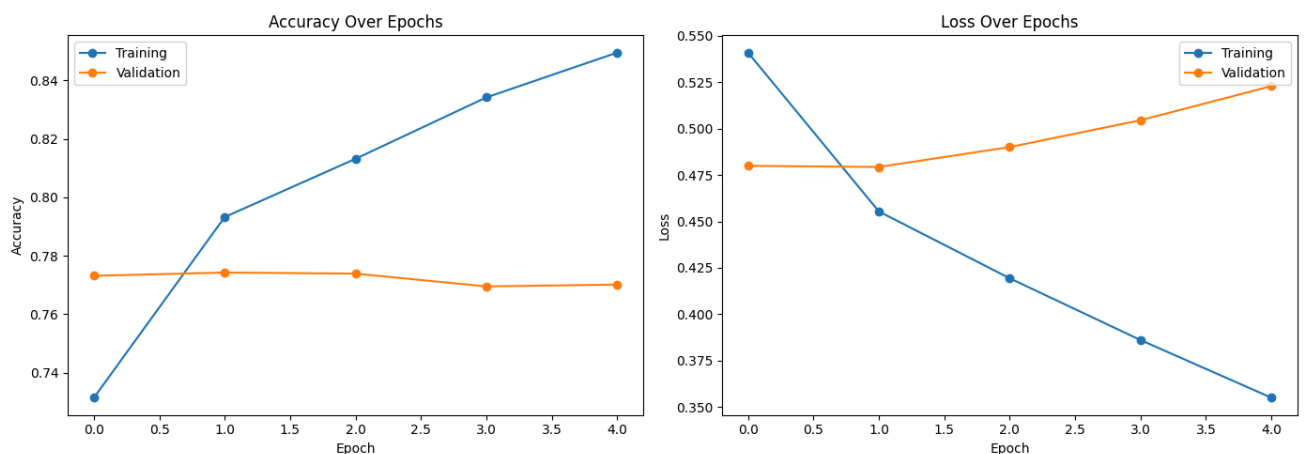
```

# Accuracy plot
axes[0].plot(history.history['accuracy'], label='Training',
axes[0].plot(history.history['val_accuracy'], label='Validation')
axes[0].set_title('Accuracy Over Epochs')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Accuracy')
axes[0].legend()

# Loss plot
axes[1].plot(history.history['loss'], label='Training', mark
axes[1].plot(history.history['val_loss'], label='Validation'
axes[1].set_title('Loss Over Epochs')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Loss')
axes[1].legend()

plt.tight_layout()
plt.show()

```



```

import joblib

# Save improved ANN
ann_improved.save('/content/drive/MyDrive/sentiment_project/

# Save improved vectorizer
joblib.dump(vectorizer_improved, '/content/drive/MyDrive/sen

print("Improved ANN and vectorizer saved! ✅")

```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file Improved ANN and vectorizer saved! ✅

Final Model Comparison Summary

Old Logistic Regression: 73.64% (20k rows)

Old Naive Bayes: 72.61% (20k rows)

Old ANN: 70.48% (20k rows)

Improved ANN: 76.81% (100k rows)

Improvement over old ANN: +6.33%

Improvement over best old model (Logistic Regression): +3.17%

What We Achieved In This Notebook

- Used 100k training data (5x more than before)
- Fixed negation problem
- Built a leaner network to reduce overfitting
- Applied early stopping to keep best weights
- Achieved 76.81% accuracy

```
from wordcloud import WordCloud

# Separate positive and negative tweets
positive_tweets = ' '.join(df[df['target'] == 1]['clean_text'])
negative_tweets = ' '.join(df[df['target'] == 0]['clean_text'])

# Plot them side by side
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Positive word cloud
wc_pos = WordCloud(width=600, height=400, background_color='white')
axes[0].imshow(wc_pos)
axes[0].set_title('Positive Tweets')
axes[0].axis('off')

# Negative word cloud
wc_neg = WordCloud(width=600, height=400, background_color='white')
axes[1].imshow(wc_neg)
axes[1].set_title('Negative Tweets')
axes[1].axis('off')

plt.show()
```

